

Spacium

A 2D LiDAR sweep in, steering and throttle out: 1,146 parameters and 2.4 kB of int8 driving a racetrack on an Arduino Nano 33 BLE Rev2.

61 → 16 → 8 → 2

7.1 M PPO steps

137 μ s / inference

1.74 laps, 0 crashes

Sam Kutsyn · 2581500 Volodymyr Kuchera · 2523181

We planned to copy a driver. It couldn't drive.

WHY RACING

This is the software half of a 1/28 Kyosho Mini-Z retrofit we want to build. Racing gives us a hard deadline and a metric nobody can argue with.

WHY 2D LIDAR, NOT A CAMERA

A camera needs a CNN we cannot flash, and a single ToF sensor cannot see far enough ahead to pick a line. A LiDAR sweep is a short 1-D vector — cheap and compressible.

WHY SIMULATED

No car and no LiDAR this quarter, so we wrote the simulator and generated the tracks. Everything else is real: real int8 weights, real board, real timing.

THE PROPOSAL SAID

Clone the expert, then compress it.

Supervised regression onto a racing line, teacher → student, int8.

WHAT ACTUALLY HAPPENED

The clone drove 0.15 laps and crashed on every layout. Copying a driver per step is not driving.

SO WE CHANGED THE TEACHER

PPO trains the small net to lap instead of imitating a controller that reads a map. The clone became its warm start; distillation and int8 stayed.

Race a car from range data alone.

TASK

Closed-loop regression: two commands every 20 ms, each one changing what the car sees next.

INPUT · 61 FLOATS

28 LiDAR beams

28 sweep deltas

4 proprioception

1 past throttle

240° field of view, 150 m range, 10 of 28 beams inside $\pm 20^\circ$.

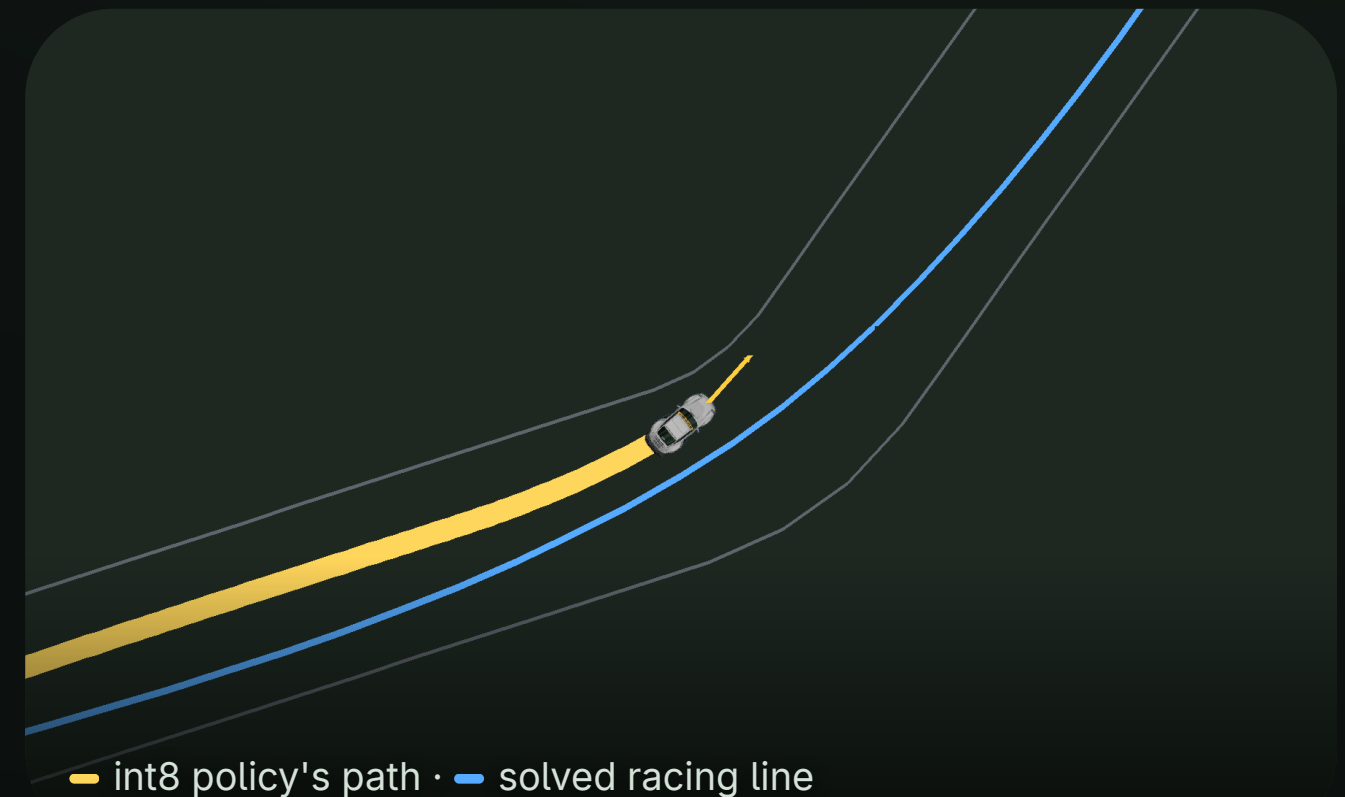
Output · 2 floats — steer, throttle / brake $\in [-1, 1]$.

EVALUATION METRIC

Judged by laps, not by loss.

Equal action MAE, opposite corners crashed.

Compression is judged by driving.



The track is the dataset.



One held-out layout, one lap by the deployed int8 policy

LAYOUT GENERATOR

Random hull → main straight → hooks and chicanes → every vertex filleted to arcs. Shape targets from 18 F1 circuits; 13 m wide, 3–6 km.

LABEL SOURCE

A minimum-curvature racing line, driven by a pure-pursuit expert on privileged state. DART noise on the action and on the label.

64

training layouts

8

held-out layouts

200,000 cloning samples · disjoint seed ranges.

The host trains. The board just drives.

Simulate

tracks, physics, 28-beam LiDAR at 50 Hz



Clone

200k expert samples, 64-64 net



PPO

7.1 M steps, 16 envs, warm-started



Distill

into the 16-8 student that ships



Quantize

post-training int8 → C header



Flash

arduino-cli, gate on ONNX + laps

DEMO-TIME LOOP

The laptop drives the simulation and ships 61 floats over USB at 500 kbaud. The board answers with **steer**, **throttle** and its own μ s timing.

Frames are checksummed, and the handshake rejects a stale flash by comparing MODEL_DIGEST against a fresh export.

FOUR COMMANDS, ONE ARTIFACT BUNDLE

tinym1-train

clone, PPO, distill → one snapshot

tinym1-build

quantize, export, compile, flash

tinym1-board

replay frames, board against host

tinym1-watch

every policy on one layout

Four policies, one deployable.

APPROACH	SEES	SHAPE	PARAMS
Pure-pursuit expert	privileged state	analytic	—
Cloned expert	61-float obs	61-64-64-2	8.4 k
PPO policy	61-float obs	61-64-64-2	8.4 k
Distilled student	61-float obs	61-16-8-2	1,146

tanh throughout, linear head clipped to $[-1, 1]$. Two hidden layers by evidence (Thomas et al., EANN 2017), width by flash. Critic 256-256, discarded at export.

ADVANCED COMPONENT 1

Cascaded compression, stage by stage
8.4 k $\rightarrow$ 1,146 params $\cdot$ int8 $\rightarrow$ 2.4 kB $\cdot$ DSP kernel
 $\rightarrow$ 137 μ s.

ADVANCED COMPONENT 2

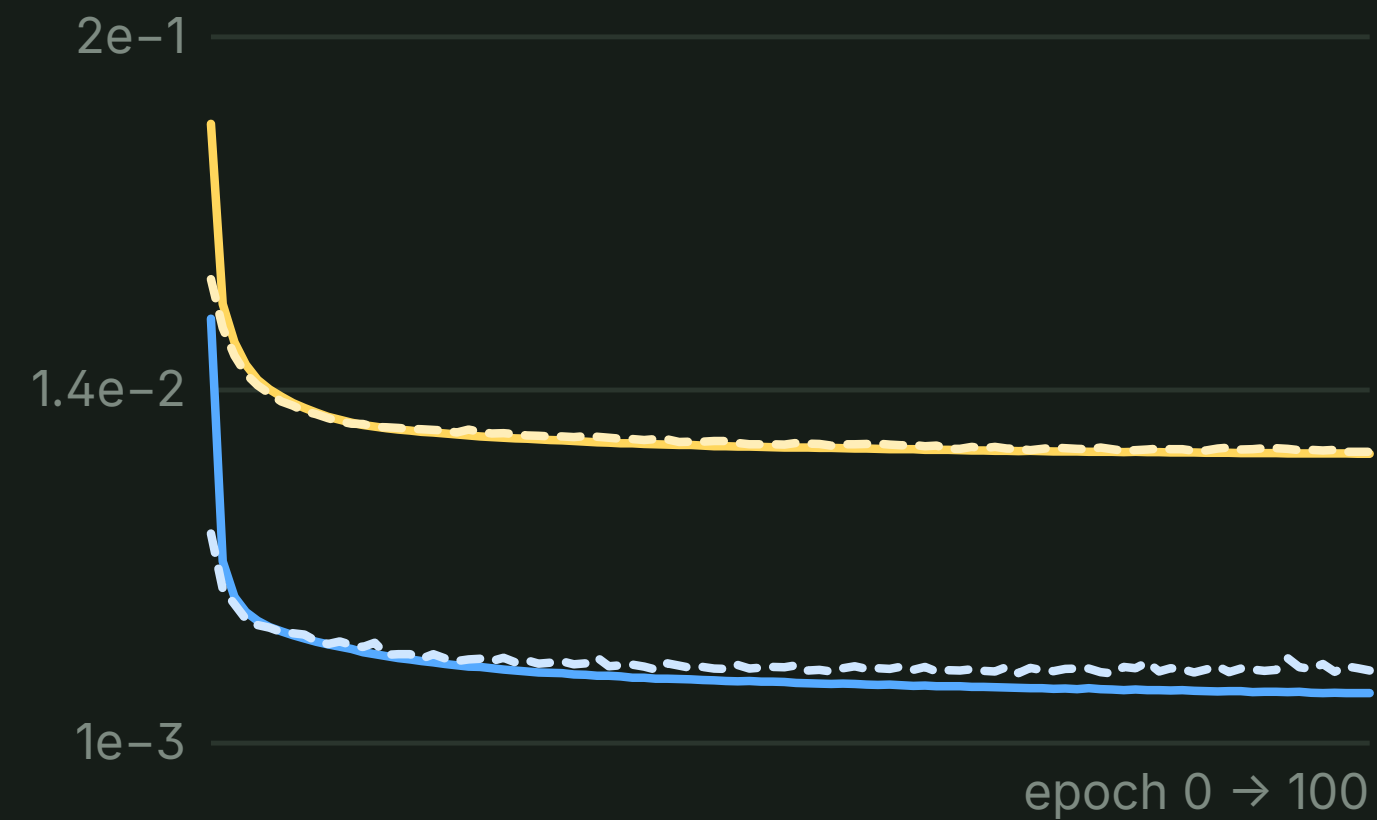
Seven drivers, identical layouts and spawns
expert $\cdot$ clone $\cdot$ PPO $\cdot$ student $\cdot$ float32 $\cdot$ int8 $\cdot$
board.

ADVANCED COMPONENT 3

Output beyond the Serial Monitor
Binary USB-CDC protocol driving a live visualizer.

Not overfitting — out of capacity.

200,000 SAMPLES · 5 % HELD-OUT VALIDATION SPLIT



— clone train

- clone val

— distill train

- distill val

STAGE	TRAIN MSE	VAL MSE	VAL / TRAIN
Clone → 64-64	0.00146	0.00172	1.18×
Distill → 16-8	0.00876	0.00889	1.01×

Per-step accuracy is not driving.

0.0017 MSE, and it still crashes on every held-out layout. Small per-step error integrates around a lap.

PPO does the actual driving.

UNCOMPRESSED POLICY	REWARD	LAPS	CRASH RATE
Cloned expert (64-64)	242 ± 397	0.15	100 %
PPO policy (64-64)	4,490 ± 1,871	1.53	40 %
Distilled student (16-8)	5,561 ± 515	1.77	0 %

5 held-out layouts, clean detector, 200 s episode budget.

SELECTED BASELINE · 8 HELD-OUT LAYOUTS × 10,000 STEPS

5,519 ± 470

reward

1.749

laps

0 %

crashes

232 km/h

top speed

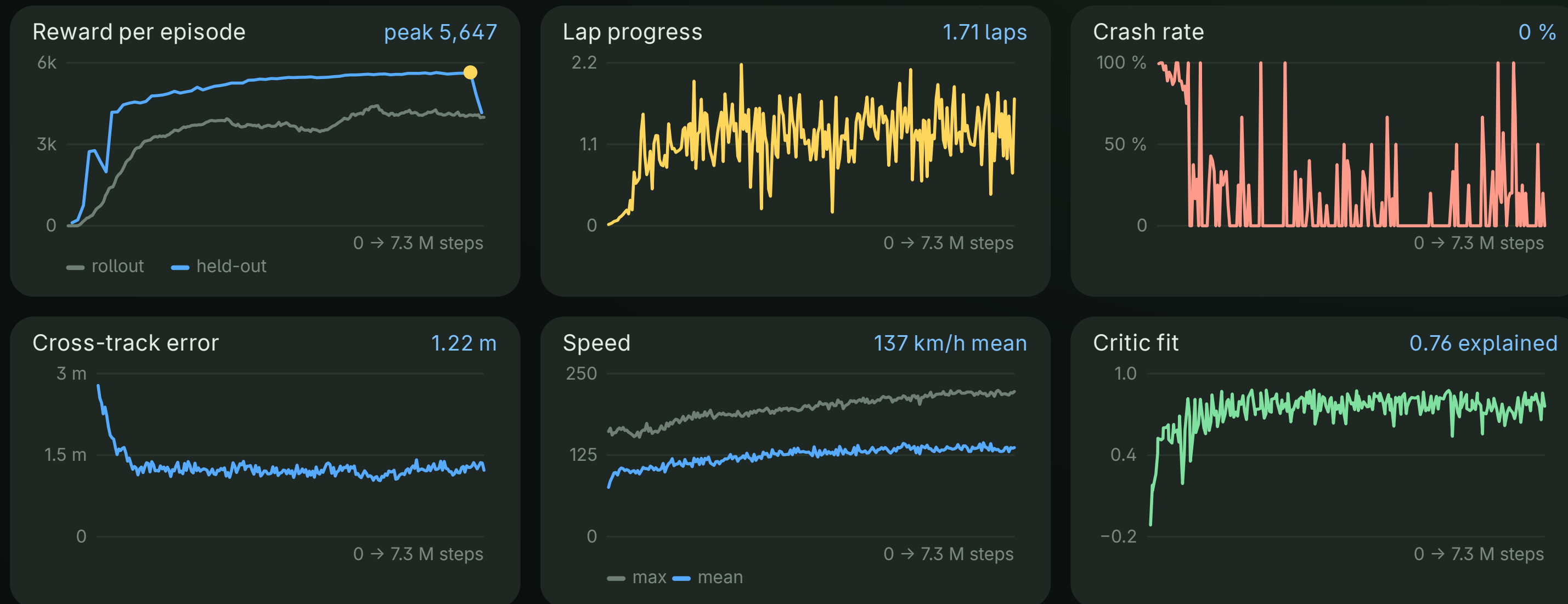
WHY THE STUDENT, NOT THE PPO POLICY

Distilling the deterministic mean of the best checkpoint into 16-8 did not just shrink the net: it removed the 40 % crash rate. The stochastic policy's exploration noise is what was putting it into walls.

EVALUATION PROTOCOL

Evaluation seeds, disjoint from training. An episode ends off-track, at 200 s, or stalled. Reward is dense progress, so 5,519 ≈ 1.75 laps.

What 7.3 million steps look like.



16 environments, 2,048-step rollouts, lr $3e-5$, clip 0.2, entropy 0.003. Warm-started from the clone, so nothing starts at zero. The dip after 7.1 M is why best.pt shipped, not the last checkpoint.

1.9× smaller. Same laps.

MODEL / CONFIGURATION	REWARD	LAPS	CRASH	ACTION MAE	FLASH (CONSTANTS)	ON-DEVICE LATENCY
Baseline float32	5,519	1.749	0 %	—	4,584 B	not deployed
int8, host emulator	5,483 -0.65 %	1.740	0 %	0.0170	2,412 B 1.90×	—
int8 on the board	5,486 -0.60 %	1.740	0 %	0.0170	2,412 B 1.90×	137 μs mean

QUANTIZATION CONFIGURATION

Post-training int8, symmetric, no zero points. Weights per output channel, activations per tensor, calibrated on 2,048 of the policy's own rollout states.

1,120 weights = 1,384 B

tanh table = 1,028 B

kernel scratch = 549 B RAM

Whole flashed sketch, mbed core included: 104,684 B of 983,040 flash (10 %) and 44,864 B of 262,144 RAM (17 %).

WHERE THE 2.4 KB GOES

float32 weights		4,584 B
int8 weights + table		2,412 B
int8 weights alone		1,384 B

Pessimistic on purpose: int8 is charged for the tanh table, float32 is not charged for libm. On weights alone the ratio is 3.31×.

The board graded itself.

ON-DEVICE OUTPUT	SAMPLES	MAE VS FLOAT32	MAX ERROR
steer	80,000 steps	0.0290	0.0969
throttle / brake	80,000 steps	0.00496	0.0969
board vs host emulator	128 frames	bit-exact	5.96e-08

HELD-OUT LAPS DRIVEN BY THE ARDUINO · 8 LAYOUTS × 10,000 STEPS

5,486 ± 469

reward, -0.60 % of float32

1.740

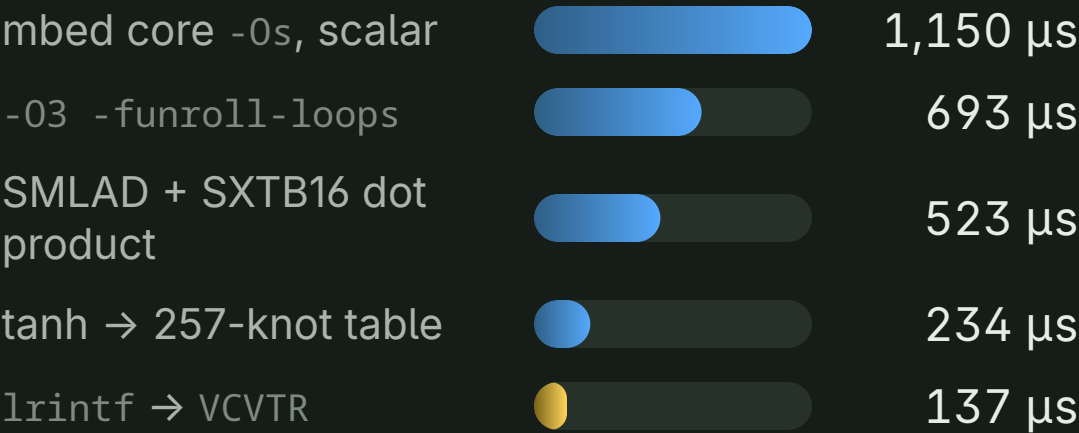
laps

0 %

crashes

Every action the car obeyed came off the board, at 137 μs — 0.7 % of the 20 ms interval. Per step it is bit-identical to the host emulator: board_verified: true.

HOW THE KERNEL GOT THERE · 1,120 MACS ON A CORTEX-M4



8.4× in total, and only the tanh table costs exactness — 84× below the int8 LSB it is requantized to. The first three rows were measured on an earlier net.

What we got, and what it taught us.

WHAT WE SHIPPED

16-8 student · int8 · DSP kernel

1.740 laps, no crashes — 2,412 B and 137 μ s on the board.

WHAT SURPRISED US

The tanh table is a fixed cost: 3.3 $\times$ on weights, 1.9 $\times$ once flashed.

CHALLENGE · BUILD, NOT MODEL

Build-level optimizations, not model changes: 1,150 $\rightarrow$ 137 μ s.

CHALLENGE · WHAT TO TRUST

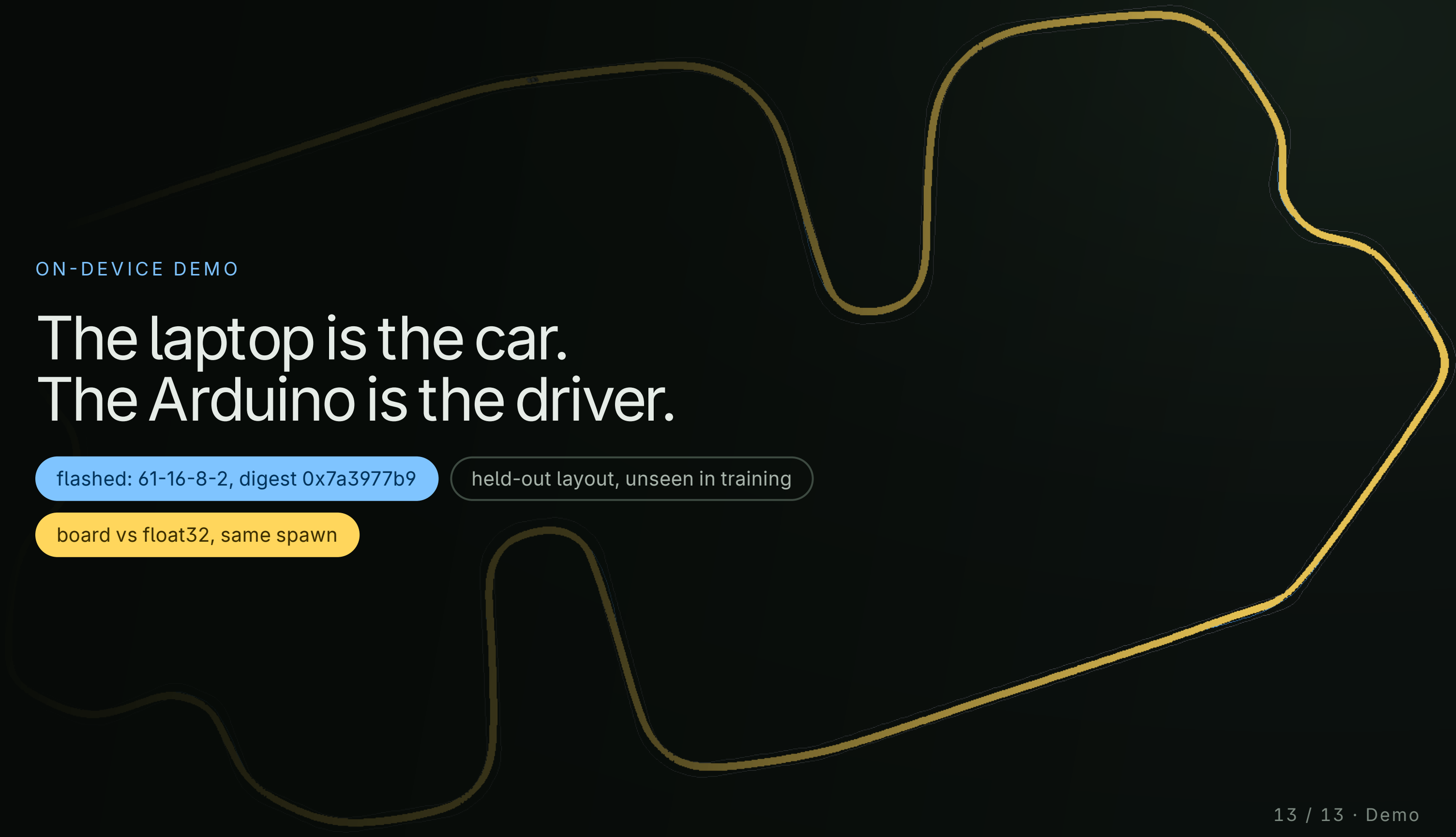
0.0017 MSE still crashed every lap, so we grade on laps and check the kernel bit-for-bit.

LIMITATIONS

LiDAR is simulated, so no sim-to-real claim. Training stays on the host; USB, not radio.

WHAT WE WOULD DO NEXT

Full-integer requantization saves $\sim 35 \mu$ s, but costs the exact-match test.



ON-DEVICE DEMO

The laptop is the car.
The Arduino is the driver.

flashed: 61-16-8-2, digest 0x7a3977b9

held-out layout, unseen in training

board vs float32, same spawn